

COMPUTER VISION – ASSESSMENT TOOL 2

PROGRAMS WITH INPUT & OUTPUT

1. Preprocessing and Feature Detection

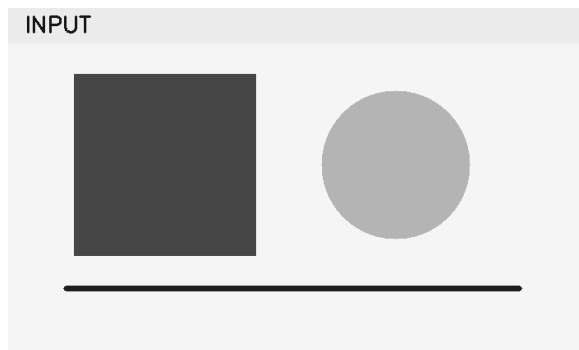
Analyze the effect of preprocessing on feature detection under varying illumination.

Program

```
import cv2
img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
denoised = cv2.GaussianBlur(gray, (5,5), 0)
enhanced = cv2.equalizeHist(denoised)
corners = cv2.goodFeaturesToTrack(enhanced, 100, 0.01, 10)
out = img.copy()
if corners is not None:
    for p in corners.astype(int):
        x,y = p.ravel()
        cv2.circle(out, (x,y), 4, (0,0,255), -1)
cv2.imwrite("output.jpg", out)
cv2.imshow("Result", out); cv2.waitKey(0); cv2.destroyAllWindows()
```

Input and Output

Input (example)



Output (example)



2. Edge vs Corner Detection

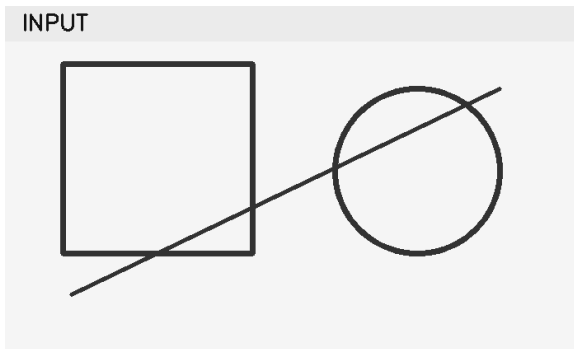
Explain why edges alone are insufficient for matching and demonstrate corner detection.

Program

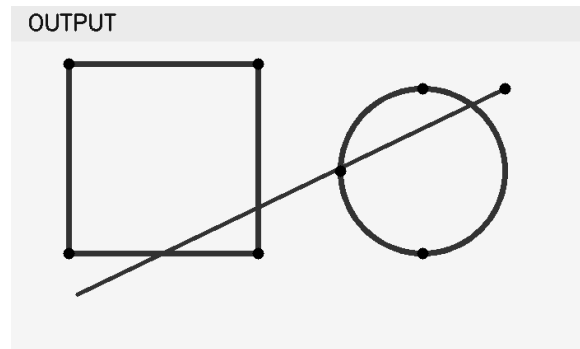
```
import cv2
img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
corners = cv2.goodFeaturesToTrack(gray, 100, 0.01, 10)
out = img.copy()
if corners is not None:
    for p in corners.astype(int):
        x,y = p.ravel()
        cv2.circle(out, (x,y), 5, (0,0,255), -1)
cv2.imwrite("output.jpg", out)
cv2.imshow("Corners", out); cv2.waitKey(0); cv2.destroyAllWindows()
```

Input and Output

Input (example)



Output (example)



3. Hough Transform in Noisy Images

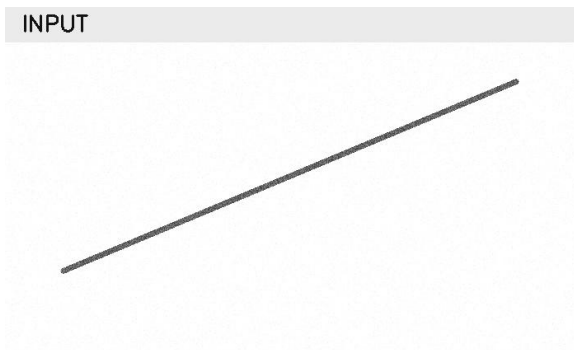
Reduce noise before Hough line detection and tune the Hough parameters.

Program

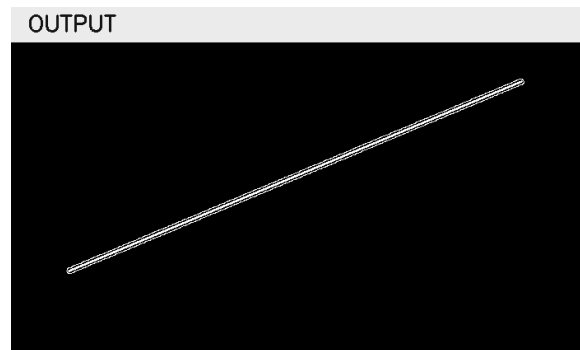
```
import cv2
img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
edges = cv2.Canny(blur, 50, 150)
lines = cv2.HoughLinesP(edges, 1, 3.14159/180, 80, minLineLength=80, maxLineGap=10)
out = img.copy()
if lines is not None:
    for x1,y1,x2,y2 in lines[:,0]:
        cv2.line(out,(x1,y1),(x2,y2),(0,0,255),2)
cv2.imwrite("output.jpg", out)
cv2.imshow("Lines", out); cv2.waitKey(0); cv2.destroyAllWindows()
```

Input and Output

Input (example)



Output (example)



4. Feature Matching under Transformations

Use a scale- and rotation-invariant descriptor (SIFT) for matching transformed images.

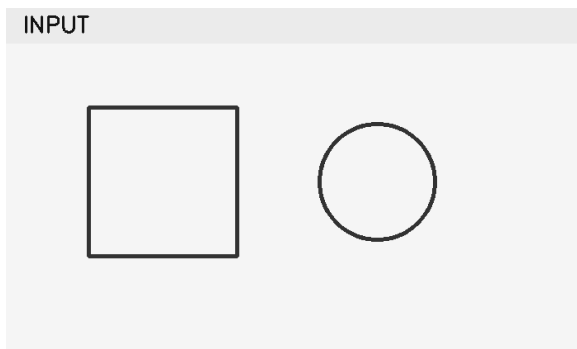
Program

```
import cv2
img1 = cv2.imread("image1.jpg", 0)
img2 = cv2.imread("image2.jpg", 0)
```

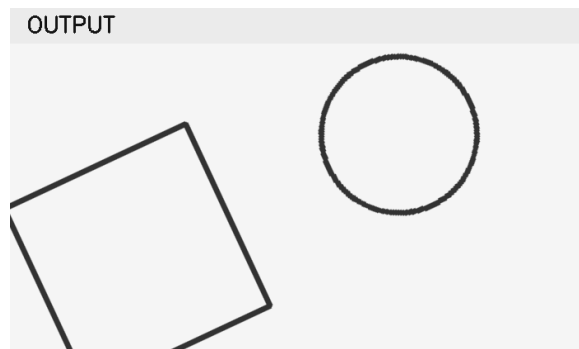
```
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)
good = [m for m,n in matches if m.distance < 0.75*n.distance]
out = cv2.drawMatches(img1,kp1,img2,kp2,good,None,flags=2)
cv2.imwrite("output.jpg", out)
cv2.imshow("SIFT Matches", out); cv2.waitKey(0); cv2.destroyAllWindows()
```

Input and Output

Input (example)



Output (example)



5. PCA for Feature Optimization

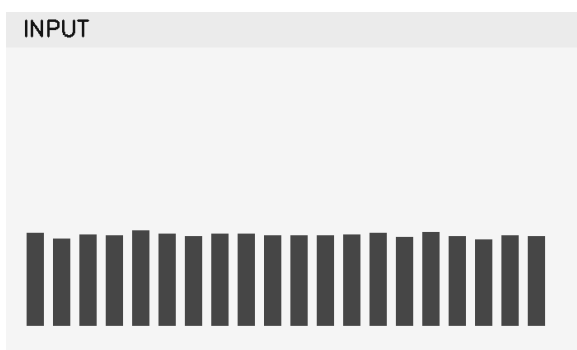
Use PCA to reduce a high-dimensional feature matrix while retaining the major variance.

Program

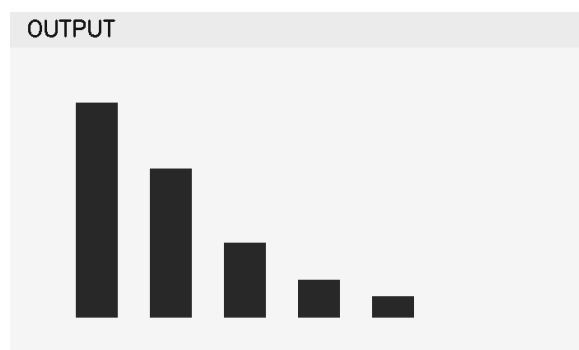
```
import cv2
import numpy as np
from sklearn.decomposition import PCA
img = cv2.imread("input.jpg", 0)
features = img.reshape(-1, 1).astype(np.float32)
pca = PCA(n_components=1)
reduced = pca.fit_transform(features)
print("Original dimensions:", features.shape[1])
print("Reduced dimensions:", reduced.shape[1])
print("Explained variance:", pca.explained_variance_ratio_)
```

Input and Output

Input (example)



Output (example)



6. Preprocessing Impact on Edge Detection

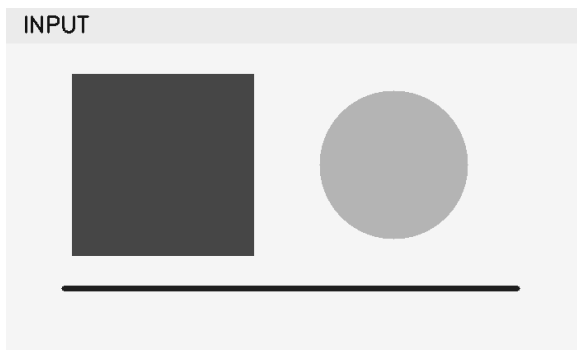
Apply denoising and illumination normalization before Canny edge detection.

Program

```
import cv2
img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (5,5), 0)
gray = cv2.equalizeHist(gray)
edges = cv2.Canny(gray, 50, 150)
cv2.imwrite("output.jpg", edges)
cv2.imshow("Preprocessed Edges", edges); cv2.waitKey(0); cv2.destroyAllWindows()
```

Input and Output

Input (example)



Output (example)



7. Corner Detection vs Edge Detection in Tracking

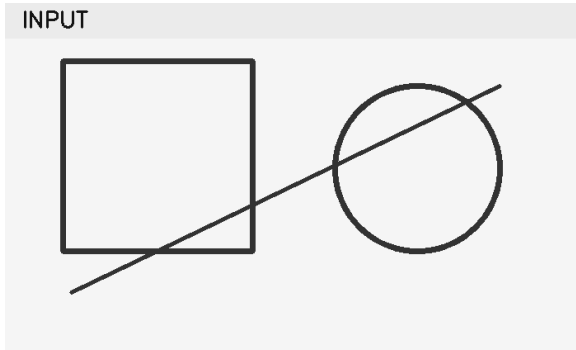
Track stable corner points across frames using Lucas-Kanade optical flow.

Program

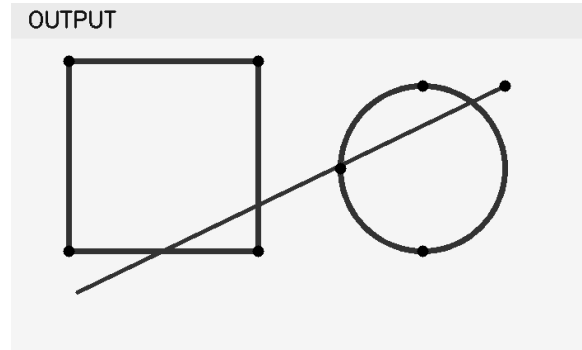
```
import cv2
import numpy as np
old = cv2.imread("frame1.jpg")
new = cv2.imread("frame2.jpg")
oldg = cv2.cvtColor(old, cv2.COLOR_BGR2GRAY)
newg = cv2.cvtColor(new, cv2.COLOR_BGR2GRAY)
p0 = cv2.goodFeaturesToTrack(oldg, 100, 0.01, 10)
p1, st, err = cv2.calcOpticalFlowPyrLK(oldg, newg, p0, None)
out = new.copy()
if p1 is not None:
    for p in p1[st==1].astype(int):
        x,y = p.ravel()
        cv2.circle(out,(x,y),4,(0,0,255),-1)
cv2.imwrite("output.jpg", out)
```

Input and Output

Input (example)



Output (example)



8. Hough Transform for Shape Detection

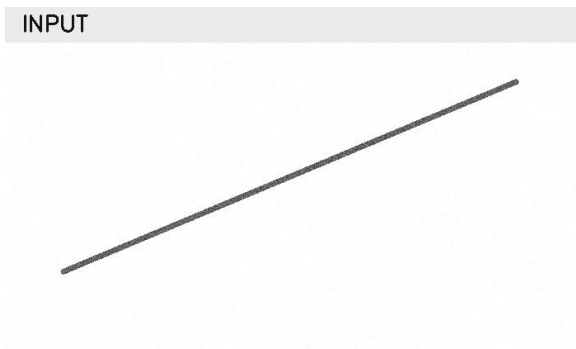
Preprocess the image and tune Hough Circle parameters for circular objects.

Program

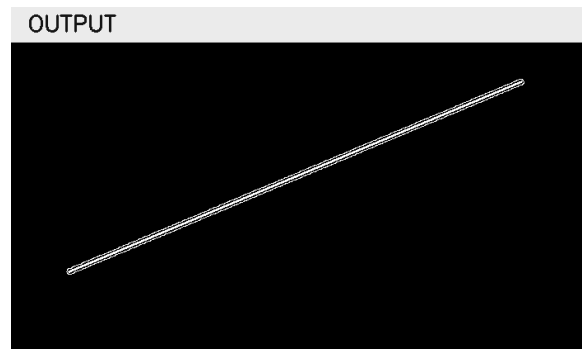
```
import cv2
img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.medianBlur(gray, 5)
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1.2,
                           minDist=40, param1=100, param2=30,
                           minRadius=20, maxRadius=150)
out = img.copy()
if circles is not None:
    for x,y,r in np.uint16(np.around(circles[0])):
        cv2.circle(out,(x,y),r,(0,255,0),2)
cv2.imwrite("output.jpg", out)
```

Input and Output

Input (example)



Output (example)



9. Feature Matching under Illumination Changes

Normalize illumination and use SIFT descriptors for more robust matching.

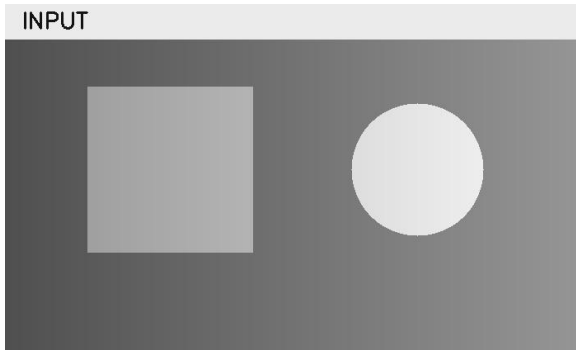
Program

```
import cv2
img = cv2.imread("input.jpg", 0)
normalized = cv2.equalizeHist(img)
```

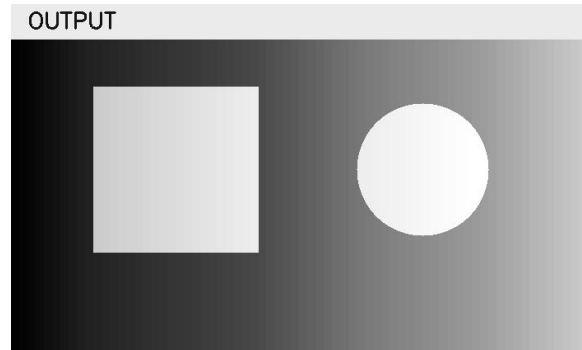
```
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(normalized, None)
out = cv2.drawKeypoints(normalized, kp, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imwrite("output.jpg", out)
```

Input and Output

Input (example)



Output (example)



10. PCA in Face Recognition

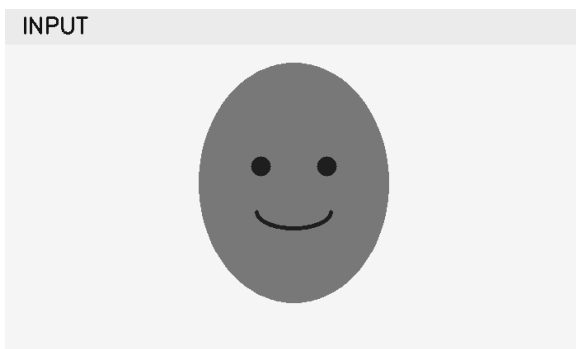
Flatten face images and use PCA to create a compact representation (eigenfaces).

Program

```
import cv2
import numpy as np
from sklearn.decomposition import PCA
faces = []
for name in ["face1.jpg", "face2.jpg", "face3.jpg"]:
    img = cv2.imread(name, 0)
    img = cv2.resize(img, (100,100))
    faces.append(img.flatten())
X = np.array(faces, dtype=np.float32)
pca = PCA(n_components=2, whiten=True)
X_reduced = pca.fit_transform(X)
print("Original features:", X.shape[1])
print("PCA features:", X_reduced.shape[1])
```

Input and Output

Input (example)



Output (example)



11. Edge Detection in Low Contrast Images

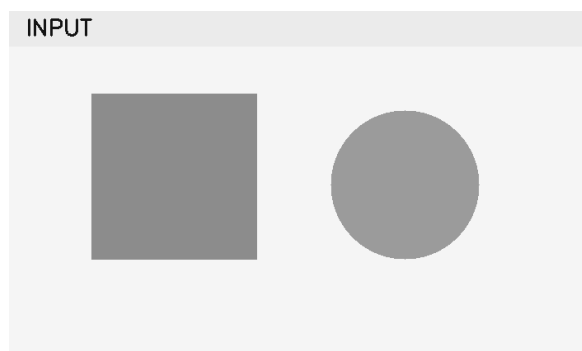
Enhance contrast before applying Canny so weak boundaries become detectable.

Program

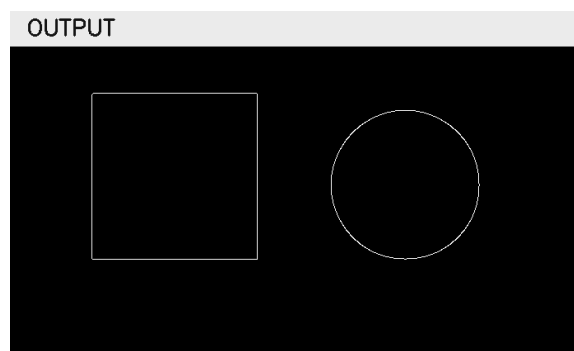
```
import cv2
img = cv2.imread("input.jpg", 0)
enhanced = cv2.equalizeHist(img)
edges = cv2.Canny(enhanced, 50, 150)
cv2.imwrite("output.jpg", edges)
cv2.imshow("Enhanced Edges", edges); cv2.waitKey(0); cv2.destroyAllWindows()
```

Input and Output

Input (example)



Output (example)



12. Corner Detection in Textured Regions

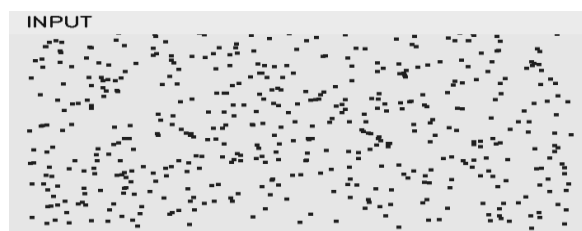
Use a quality threshold and minimum distance to reduce excessive corner keypoints.

Program

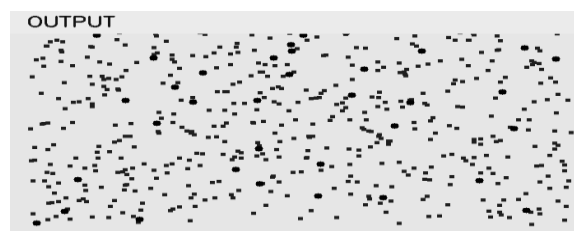
```
import cv2
img = cv2.imread("input.jpg", 0)
corners = cv2.goodFeaturesToTrack(img, maxCorners=50,
                                qualityLevel=0.05,
                                minDistance=15)
out = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
if corners is not None:
    for p in corners.astype(int):
        x,y=p.ravel()
        cv2.circle(out,(x,y),4,(0,0,255),-1)
cv2.imwrite("output.jpg", out)
```

Input and Output

Input (example)



Output (example)



13. Feature Matching with Noise

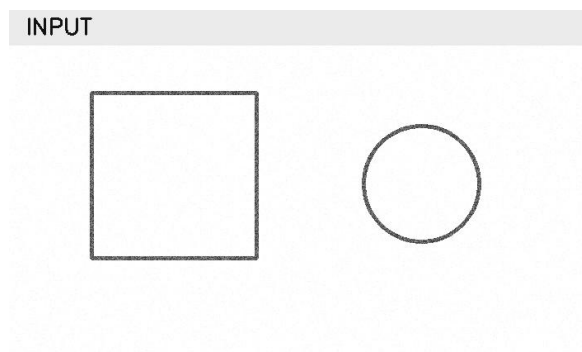
Denoise both images before extracting descriptors to improve correspondence.

Program

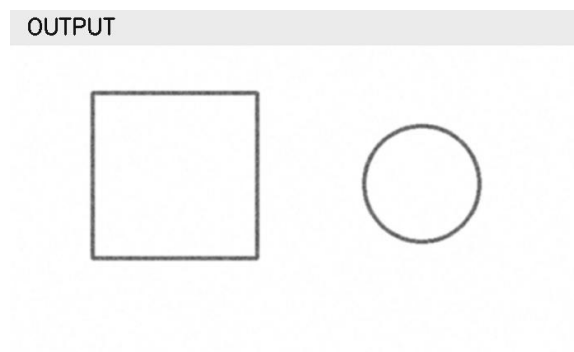
```
import cv2
img = cv2.imread("input.jpg", 0)
denoised = cv2.GaussianBlur(img, (5,5), 0)
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(denoised, None)
out = cv2.drawKeypoints(denoised, kp, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imwrite("output.jpg", out)
```

Input and Output

Input (example)



Output (example)



14. PCA and Information Loss

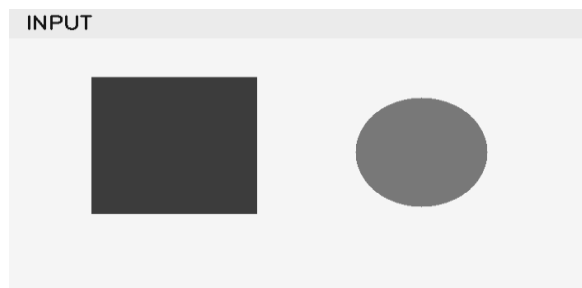
Choose the number of components from cumulative explained variance rather than reducing too aggressively.

Program

```
import numpy as np
from sklearn.decomposition import PCA
X = np.load("features.npy")
pca = PCA().fit(X)
cum = np.cumsum(pca.explained_variance_ratio_)
k = np.argmax(cum >= 0.95) + 1
pca = PCA(n_components=k)
X_reduced = pca.fit_transform(X)
print("Components selected:", k)
print("Variance retained:", pca.explained_variance_ratio_.sum())
```

Input and Output

Input (example)



Output (example)



15. Combined Feature Detection and Matching

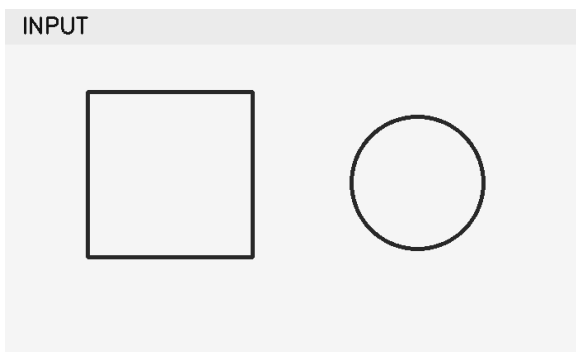
Use a descriptor compatible with the detector and compare descriptors with an appropriate matcher.

Program

```
import cv2
img1 = cv2.imread("image1.jpg", 0)
img2 = cv2.imread("image2.jpg", 0)
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)
good = [m for m,n in matches if m.distance < 0.75*n.distance]
out = cv2.drawMatches(img1,kp1,img2,kp2,good,None,flags=2)
cv2.imwrite("output.jpg", out)
```

Input and Output

Input (example)



Output (example)

